Color Tracking (PTZ)

1. Experiment Objective

The PTZ pan-tilt tracks an object of a specified color, keeping the target at the center of the frame.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Tracking	Detects and continuously locks onto a target through a vision algorithm; only the pan-tilt follows, the car does not move
PTZ Pan-Tilt-Zoom	A two-degree-of-freedom servo pan-tilt; s1 rotates horizontally and s2 tilts, controlled via the `/cmd_ptz_angle` topic (`geometry_msgs/Vector3`)
HSV Color Space	Describes colors with the three dimensions Hue, Saturation, and Value; better suited than RGB for color threshold filtering and target segmentation
PD Control	Proportional-derivative control; computes the pan-tilt angle step from the target offset (P term) and the offset change rate (D term), with no integral term to avoid overshoot
Deadband	When the target offset is within the deadband pixel range, the pan-tilt does not move, avoiding continuous jitter caused by tiny errors

2. Technical Architecture

The color tracking node `color_tracker_ptz_node` subscribes to the raw image topic `/camera/image_raw` of the ESP32 camera, segments the target color region with HSV thresholding, computes the pan-tilt angle step with a PD controller, and publishes `geometry_msgs/msg/Vector3` to `/cmd_ptz_angle` to control the pan-tilt servos to track the target.

Detection pipeline: image conversion → HSV conversion → inRange binarization → morphological closing → findContours largest contour extraction → minEnclosingCircle target center → EMA low-pass filter → PD pan-tilt control (fixed 25 Hz).

3. Core Topics Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/msg/Image	best_effort	Raw image from the ESP32 camera

Firmware Downlink (Control Commands)

Topic	Message Type	QoS	Field	Range	Direction
/cmd_ptz_angle	geometry_msgs/msg/Vector3	best_effort + keep_last(1)	x	[-90, 90]°	s1 horizontal servo angle
			y	[-90, 20]°	s2 tilt servo angle

ROS2 Internal Topics

Topic	Message Type	QoS	Source	Description
/color_tracker_ptz/debug_image	sensor_msgs/msg/Image	reliable	color_tracker_ptz_node	Debug image (published when `publish_debug_image=true`)

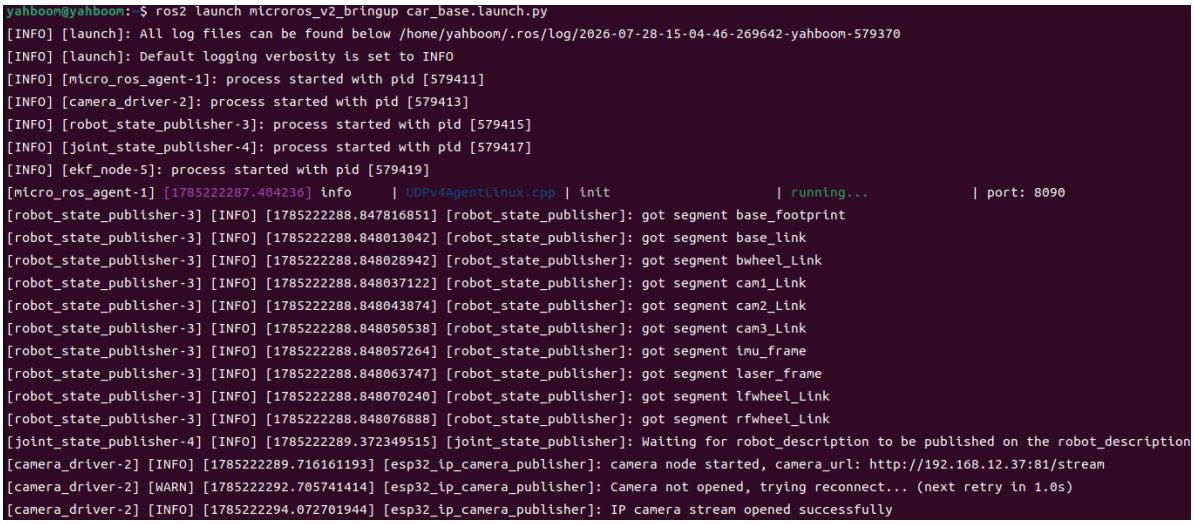
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑ Supported
No-PTZ version	❌
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

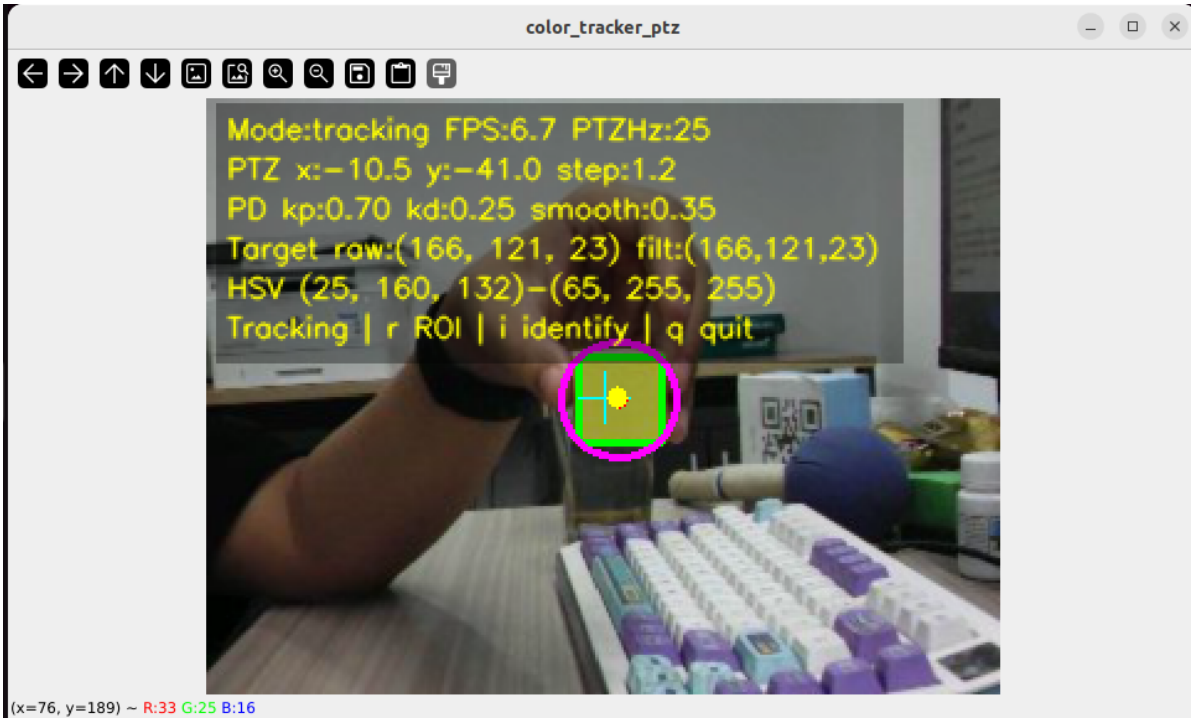
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start Color Tracking (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg color_tracker_ptz.launch.py
```



4. Operation Instructions

Workflow

After startup, the node receives camera frames at the image frame rate and executes the following steps on every processing cycle:

1. Image decoding and preprocessing

The node subscribes to the raw image topic `/camera/image_raw` (QoS: BEST_EFFORT, KEEP_LAST, depth=1), converts it to a BGR frame with `CvBridge.imgmsg_to_cv2(..., desired_encoding="bgr8")`, scales it to `image_width` × `image_height` (320×240 by default), and then enters the processing pipeline.

2. HSV color detection

The BGR frame is converted to the HSV color space and binarized with `cv.inRange` using the six-parameter threshold `Hmin/Smin/Vmin ~ Hmax/Smax/Vmax`, producing a mask of the target color region. A rectangular kernel (`morph_kernel_size`, 5×5 by default) then performs a morphological closing (`MORPH_CLOSE`) to fill holes and gaps, and the result is threshold-binarized again to generate the final mask.

3. Target extraction and filtering

`cv.findContours` (`RETR_EXTERNAL`) finds external contours in the mask; the contour with the largest `cv.contourArea` is taken as the candidate target. Its `cv.minAreaRect` rotated rectangle is computed, then `cv.minEnclosingCircle` obtains the minimum enclosing circle (center cx/cy and radius). The target is marked valid only when the radius is ≥ `min_target_radius` (8 px) and the area is ≥ `min_target_area` (100 px²).

4. Target center low-pass filter

The valid target center is passed through a first-order low-pass filter: `filtered = (1-α) × old + α × new`, with α = `target_smooth_alpha` (0.6). The first detected target uses the raw value directly; later frames transition smoothly, reducing center jitter from OpenCV detection noise.

5. PD pan-tilt control (fixed 25 Hz)

A fixed-rate control timer (`ptz_control_rate` = 25 Hz), independent of image processing, runs the PD control and outputs only in the `tracking` state:

- Compute the pixel error: `error = target_center - image_center`
- Deadband check: if `|error| < deadband` (20 px), zero the error
- Normalize: `normalized_error = error / half_image_size`
- PD output: `raw_step = (kp × normalized_error + kd × d(normalized_error)/dt) × max_ptz_step`
- Step limiting: clamp `raw_step` to `[-max_ptz_step, +max_ptz_step]` (±1.2°)
- Step smoothing: `step = (1-α) × last_step + α × raw_step` (α = `ptz_step_smooth_alpha` = 0.35)
- Angle smoothing: `angle = (1-α) × current_angle + α × desired_angle` (α = `ptz_smooth_alpha` = 0.85)
- Publish `geometry_msgs/Vector3` to `/cmd_ptz_angle` after angle limiting (x = horizontal, y = tilt)

6. Color sampling (ROI box selection)

Press `r` to enter ROI sampling mode (state `init`) and drag the mouse to box-select the target color region. On mouse release, the node computes the min/max values of each HSV channel inside the box and adds an expansion margin (`h_expand`=5, `s_expand`=20, `v_expand`=20). The sampled result is written to both the ROS parameters and the persistent file `color_hsv.txt`, then the node switches to recognition mode.

Interaction

Key	Function	Description
Space	Tracking/recognition switch	Toggles between "recognition mode" (detects only, does not control the pan-tilt) and "tracking mode" (detects + controls the pan-tilt)
r / R	ROI color sampling	Enters sampling mode; drag the mouse to box-select the target color region and extract the HSV threshold automatically
i / I	Recognition mode	Only detects the target and overlays it on the frame, without publishing pan-tilt control commands
q / Q / ESC	Exit	Force-exits the node process with `os._exit(0)`

Mouse operation: In ROI sampling mode, hold the left button and drag to box-select the target color region (at least 3×3 pixels); on release, the HSV threshold is sampled automatically and saved persistently.

On-screen overlay: The debug window overlays the current mode, frame rate, pan-tilt angle, PD parameters, HSV threshold range, target state (raw/filtered/lost), and key hints in real time.

Safety and Edge Cases

- **No valid HSV threshold**: At startup, if there is no HSV config file and `auto_start_tracking=false`, the node enters the `init` state automatically and waits for the user to press `r` to box-select and sample a color
- **HSV file exists**: At startup, the HSV threshold is loaded automatically from `color_hsv.txt`; if `auto_start_tracking=true` as well, the node enters tracking mode directly
- **Target lost handling**: When no valid target is detected for more than `lost_timeout` (1.0 s), the target is marked lost, the PD control state resets (error/step history cleared), and the pan-tilt stops (stays at its last position); if `lost_target_stop=true` (default), the filtered target state is also cleared
- **Pan-tilt angle limiting**: The horizontal angle is clamped to [`ptz_x_min`, `ptz_x_max`] (-90°~90°) and the tilt angle to [`ptz_y_min`, `ptz_y_max`] (-90°~20°); out-of-range values are truncated automatically
- **Single-step limiting**: The raw PD step is clamped to [-`max_ptz_step`, +`max_ptz_step`] (±1.2°) to prevent large abrupt changes
- **Deadband protection**: When the target offset is within the deadband range, the normalized error and step are forced to zero, producing no control output
- **Runtime parameter validation**: When parameters are modified via `ros2 param set /color_tracker_ptz_node <parameter> <value>` (or rqt), full range validation is performed (HSV channels [0,179]/[0,255], alpha [0.01,1.0], angle min<max, etc.); illegal values are rejected. Changes to topic parameters such as `image_topic` / `ptz_topic` / `window_name` are rejected with a prompt to restart the node
- **Exit strategy**: Press q/ESC to exit directly with `os._exit(0)`, without relying on `rclpy.shutdown`, to avoid OpenCV/ROS2 hanging during cleanup

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **No HSV threshold and no box selection**: Enters the `init` state; the frame prompts the user to press `r` to box-select the target color region, and the pan-tilt does not move
- **After box-selecting a color (recognition mode)**: The frame shows the target detection result in real time (green rotated rectangle + purple enclosing circle + red center point), overlays the HSV threshold and state, but does not publish pan-tilt control commands
- **Target detected + tracking mode**: The pan-tilt follows the target smoothly with PD control; a yellow crosshair is drawn at the frame center, the filtered target center is a solid yellow circle, and the actual pan-tilt angle and PD parameters are overlaid in real time
- **Target within the deadband**: The pixel error is smaller than the deadband (20 px), the step is forced to zero, and the pan-tilt outputs nothing, avoiding micro-jitter
- **Target briefly lost (<1 s)**: The filtered position of the last valid target is retained (the target center is not cleared), preserving tracking continuity; the pan-tilt has brief inertia
- **Target lost by timeout (>1 s)**: The PD state resets, the filtered target is cleared, and the pan-tilt stays at its position before the loss, waiting for the target to reappear
- **Press r to re-sample**: Enters ROI box-selection mode with a green drag rectangle on the frame; on release, the HSV range is extracted automatically (with expansion margin) and persisted to `color_hsv.txt`, then switches to recognition mode

5. Program Analysis

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/color_tracker_ptz.launch.py`
> - Node: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/color_tracker_ptz_node.py`

1. Launch Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/color_tracker_ptz.launch.py`
> - YAML: `~/microros_ws/src/microros_v2_ptz_visual_control_pkg/config/color_tracker_ptz.yaml`

Parameter	Type	Default Value	Description
`image_topic`	string	`/camera/image_raw`	Input raw image topic
`ptz_topic`	string	`/cmd_ptz_angle`	Pan-tilt Vector3 control topic
`hsv_file`	string	`config/color_hsv.txt`	HSV threshold file path
`display_debug_window`	bool	`true`	Show the OpenCV debug window
`auto_start_tracking`	bool	`false`	Start tracking immediately after launch when HSV is available
`enable_rqt_dynamic_parameters`	bool	`true`	Enable runtime dynamic tuning via rqt/ros2 param
`config_file`	string	`config/color_tracker_ptz.yaml`	YAML config file path
`pd_kp`	float	`0.7`	PD proportional gain
`pd_kd`	float	`0.25`	PD derivative gain
`ptz_control_rate`	float	`25.0` Hz	Pan-tilt control publishing rate (YAML override)
`deadband_x`	float	`20.0` px	Horizontal deadband pixel range
`deadband_y`	float	`20.0` px	Vertical deadband pixel range
`target_smooth_alpha`	float	`0.6`	Target center low-pass filter coefficient (YAML override)
`ptz_smooth_alpha`	float	`0.85`	Pan-tilt angle low-pass filter coefficient (YAML override)
`max_ptz_step`	float	`1.2`°	Maximum single pan-tilt angle step
`ptz_step_smooth_alpha`	float	`0.35`	Step EMA smoothing coefficient
`lost_timeout`	float	`1.0` s	Target lost timeout
`Hmin` / `Hmax`	int	`0` / `9`	HSV hue lower/upper bound [0, 179]

Parameter	Type	Default Value	Description
`Smin` / `Smax`	int	`85` / `253`	HSV saturation lower/upper bound [0, 255]
`Vmin` / `Vmax`	int	`126` / `253`	HSV value lower/upper bound [0, 255]
`min_target_radius`	float	`8.0` px	Minimum valid target radius (YAML override)
`min_target_area`	float	`100.0` px²	Minimum valid target area (YAML override)
`ptz_init_x` / `ptz_init_y`	float	`0.0`° / `-45.0`°	Initial pan-tilt horizontal/tilt angle (YAML overrides ptz_init_y)
`ptz_x_min` / `ptz_x_max`	float	`-90.0`° / `90.0`°	Horizontal angle range
`ptz_y_min` / `ptz_y_max`	float	`-90.0`° / `20.0`°	Tilt angle range

2. Core Node Analysis

`ColorTrackerPtzNode` inherits from `rclpy.node.Node` and is the core implementation of color tracking. The constructor and the PD control callback are shown below.

Constructor `__init__`

The constructor declares the ROS2 parameters, creates the publishers and subscribers, initializes the tracking state, and sets up the fixed-rate control timer:

```
class ColorTrackerPtzNode(Node):
    """ROS2 color tracker PTZ node."""

    def __init__(self) -> None:
        super().__init__("color_tracker_ptz_node")

        self.cv_bridge = CvBridge()
        self.target_state_lock = threading.Lock()

        self._declare_ros_parameters()
        self._load_ros_parameters()

        # Create publishers: pan-tilt angle command / debug image
        self.ptz_angle_publisher = self.create_publisher(Vector3,
self.ptz_topic, 10)
        self.debug_image_publisher = self.create_publisher(Image,
self.debug_image_topic, 10)

        # Create subscriber: input image (default raw image topic)
        subscribed_image_message_type = Image
        self.image_subscriber = self.create_subscription(
            subscribed_image_message_type,
```

```
            self.image_topic,
            self.image_callback,
            QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                       reliability=QoSReliabilityPolicy.BEST_EFFORT,
                       durability=QoSDurabilityPolicy.VOLATILE),
        )

        # Tracking state initialization
        self.tracking_state = "identify"

        # Load the HSV threshold (prefer loading from the file)
        self.hsv_threshold_range = self._get_hsv_range_from_parameters()
        saved_hsv = read_hsv_threshold_file(self.hsv_file)
        if saved_hsv is not None:
            self.hsv_threshold_range = saved_hsv
            self._apply_hsv_range_to_ros_parameters(saved_hsv)
            self.get_logger().info(f"Loaded HSV from file: {self.hsv_file}, "
                                   f"hsv={saved_hsv}")

        # Initial pan-tilt angles
        self.current_ptz_horizontal_angle = float(self.ptz_init_x)
        self.current_ptz_vertical_angle = float(self.ptz_init_y)

        # PD control state initialization
        self.previous_error_x = 0.0
        self.previous_error_y = 0.0
        self.previous_control_time = 0.0
        self.last_step_x = 0.0
        self.last_step_y = 0.0

        # Target detection state
        self.raw_target_circle = (0, 0, 0)
        self.filtered_target_center_x = None
        self.filtered_target_center_y = None

        # Create the fixed-rate pan-tilt control timer (25 Hz by default)
        self._recreate_ptz_control_timer(False)

        # Create the OpenCV debug window
        if self.display_debug_window:
            cv.namedWindow(self.window_name, cv.WINDOW_NORMAL)
            cv.resizeWindow(self.window_name, self.window_width,
self.window_height)
            cv.setMouseCallback(self.window_name, self._on_mouse_event)

        # Register the runtime dynamic tuning callback
        self.parameter_callback_handle = self.add_on_set_parameters_callback(
            self._handle_runtime_parameter_update
        )
```

Pan-tilt control callback `_control_timer_callback`

Triggered at a fixed rate (25 Hz); runs the PD pan-tilt control only in the `tracking` state:

```
def _control_timer_callback(self) -> None:
    """Run fixed-rate PTZ control."""
    if self.tracking_state != "tracking":
```

```
        return

    with self.target_state_lock:
        target_is_valid = self.target_is_valid
        target_age = time.perf_counter() - self.last_valid_target_time_seconds
        filtered_x = self.filtered_target_center_x
        filtered_y = self.filtered_target_center_y
        filtered_r = self.filtered_target_radius

    if (not target_is_valid or filtered_x is None
            or filtered_y is None or filtered_r is None
            or target_age > self.lost_timeout):
        self._handle_lost_target()
        return

    self._track_target_with_smooth_ptz(
        (int(filtered_x), int(filtered_y), int(filtered_r))
    )
```

PD tracking method `_track_target_with_smooth_ptz`

Implements the complete PD control chain: pixel error computation → deadband handling → normalization → PD step → step smoothing → angle smoothing → clamped publish:

```
def _track_target_with_smooth_ptz(self, target_circle: TargetCircleInPixels) ->
None:
    target_center_x, target_center_y, _ = target_circle
    image_cx = self.image_width * 0.5
    image_cy = self.image_height * 0.5

    # Compute the pixel error, zeroed inside the deadband
    err_x = float(target_center_x - image_cx)
    err_y = float(target_center_y - image_cy)
    if abs(err_x) < self.deadband_x:
        err_x = 0.0
    if abs(err_y) < self.deadband_y:
        err_y = 0.0

    # Time delta (for the derivative term)
    now = time.perf_counter()
    dt = max(now - self.previous_control_time, 1.0e-3)

    # Normalized error + derivative
    norm_x = err_x / image_cx
    norm_y = err_y / image_cy
    dx = (norm_x - self.previous_error_x) / dt
    dy = (norm_y - self.previous_error_y) / dt

    # PD computation + step limiting
    raw_x = (self.pd_kp * norm_x + self.pd_kd * dx) * self.max_ptz_step
    raw_y = (self.pd_kp * norm_y + self.pd_kd * dy) * self.max_ptz_step
    raw_x = clamp_numeric_value(raw_x, -self.max_ptz_step, self.max_ptz_step)
    raw_y = clamp_numeric_value(raw_y, -self.max_ptz_step, self.max_ptz_step)

    # Step EMA smoothing
    alpha = self.ptz_step_smooth_alpha
    step_x = (1.0 - alpha) * self.last_step_x + alpha * raw_x
```

```
    step_y = (1.0 - alpha) * self.last_step_y + alpha * raw_y

    # Save the control history
    self.previous_error_x = norm_x
    self.previous_error_y = norm_y
    self.previous_control_time = now
    self.last_step_x = step_x
    self.last_step_y = step_y

    # Compute the desired angle (with direction reversal)
    desired_h = (self.current_ptz_horizontal_angle - step_x
                 if self.ptz_x_reverse else self.current_ptz_horizontal_angle +
step_x)
    desired_v = (self.current_ptz_vertical_angle + step_y
                 if self.ptz_y_reverse else self.current_ptz_vertical_angle -
step_y)
    desired_h = clamp_numeric_value(desired_h, self.ptz_x_min, self.ptz_x_max)
    desired_v = clamp_numeric_value(desired_v, self.ptz_y_min, self.ptz_y_max)

    # Angle EMA smoothing
    ptz_alpha = self.ptz_smooth_alpha
    self.current_ptz_horizontal_angle = (
        (1.0 - ptz_alpha) * self.current_ptz_horizontal_angle + ptz_alpha *
desired_h
    )
    self.current_ptz_vertical_angle = (
        (1.0 - ptz_alpha) * self.current_ptz_vertical_angle + ptz_alpha *
desired_v
    )

    # Clamp again and publish
    self.current_ptz_horizontal_angle = clamp_numeric_value(
        self.current_ptz_horizontal_angle, self.ptz_x_min, self.ptz_x_max)
    self.current_ptz_vertical_angle = clamp_numeric_value(
        self.current_ptz_vertical_angle, self.ptz_y_min, self.ptz_y_max)
    self._publish_ptz_angle(self.current_ptz_horizontal_angle,
                            self.current_ptz_vertical_angle)
```

6. Frequently Asked Questions

Problem	Cause	Solution
Pan-tilt jitters	`pd_kp` too large or `target_smooth_alpha` too small	Reduce `pd_kp` (e.g., 0.3), increase `target_smooth_alpha` (e.g., 0.5)
Target lost	Moving too fast for the pan-tilt response, or the HSV threshold mismatches	Slow down the target; press `r` to box-select and re-sample the color
Target not detected	HSV threshold inaccurate or large lighting changes	Press `r` and re-box-select the target color region under the current lighting
Cannot enter tracking mode	No HSV sampling performed or `auto_start_tracking=false`	Press `r` to box-select, then press Space to enter tracking mode
Pan-tilt does not return to center after startup	The initial angle is set by `ptz_init_x` / `ptz_init_y`	Adjust the initial angle in the YAML or launch parameters